ADR 0130 — A claimed number is provisional until it merges

- **Status:** Accepted — implemented, mutation-proved two ways failing differently
- **Date:** 2026-09-06
- **Issue:** #689
- **Supersedes / superseded by:** —

Context

Five ADR-number collisions landed in one day across six concurrent lanes (2026-09-06), even though every lane's brief carried an explicit "check the index immediately before claiming" warning:

Number	Collided between	How it was caught
0121	#669's token storage vs #667's animation ADR	merge conflict on `docs/adr/README.md`
0125	#669 (token storage) vs #675 (port-claim lock)	grok's review of #669
0126	filename/index said 0126, the document's own H1 still said 0125	codex-daybreak, auditing something else entirely
0127	#666's ADR claimed it after another lane's brief was written	C2, merging main down
0128	#684 (clone containment) vs #584 (settings surface)	C2, merging main down

The process is "read `docs/adr/README.md` on current main, take the next free number." That is not atomic across lanes working 30-90 minutes apart: a lane reads the index at time T, works for the better part of an hour, and a faster lane has already taken the same number by T+10. Following the process correctly does not close this window, because the window is the gap between

reading the index and *merging* — and nothing on `main` reserves a number for the time in between.

0126 is a different bug wearing the same symptom. It was not two lanes claiming one number; it was a single rename that updated the filename, the index row, and cross-references, but left the document's own `# ADR NNNN` heading pointing at the old number, because the heading edit never got staged. `crates/git-vista-server/tests/adr_index_matches_the_files.rs` (#578) already caught the filename/index/row triple going out of sync; it said nothing about the fourth place the number lives — the heading inside the file itself.

Both bugs share one property worth naming: the existing `adr_index_matches_the_files` test already turns a same-number collision between two *different* files into a hard `panic!` (`files_by_number()`'s `if let Some(previous) = out.insert(...)` branch). Every collision in the table above **was already caught** — by that test at CI, or by a merge conflict, or by a reviewer. The failure is not silence; it is lateness. By the time the duplicate surfaces, the document is written, mutation-proved, and often PDF-rendered under the wrong number, and untangling that touches five places by hand: the filename, the H1, the index row, every cross-reference, and the tracked PDF.

Decision

Two additive changes. Neither touches the numbering scheme itself — sequential numbers, claimed by reading the index — because every alternative that removes the scheme costs more than the collisions do (see Alternatives). Instead, one change makes the *lateness* the only clean outcome (a mismatch fails loud and immediately, not later inspection), and the other makes the mandatory renumber cheap enough that "no collision should ever reach `main`" stops being a load-bearing property of the number-reading step.

1. A file's H1 heading is derived from its own filename, tested

`every_adr_heading_names_its_own_filename_number` (added to the existing `adr_index_matches_the_files.rs`, alongside its three siblings from #578) reads every `docs/adr/NNNN-*.md`'s first line, extracts whichever of the corpus's two heading styles it uses (`# ADR NNNN — Title`, used by 96 files, or the older `# NNNN — Title`, used by 33), and asserts the number it names agrees with the number the filename already claims. A heading with no recognisable number in either style is a failure too, not a silent pass — the third case #689's own audit found zero of, but a test that only checks the two known-good shapes would have nothing to say about a third, unrecognised one.

This is deliberately narrow: it does not touch numbering, contention, or which number is "next." It converts one specific, already-happened bug (0126) from "found by a lane auditing something unrelated" into "found by `cargo test`, immediately, on the commit that introduced the drift."

2. A renumber is one script, not five hand-edits

`scripts/adr-renumber.sh OLD NEW` renames the file, rewrites its own H1 in place (whichever of the two heading styles it uses), renames the tracked PDF twin if one exists under `docs/adr/pdf/`, updates the exact `[OLD](OLD-slug.md)` link in `docs/adr/README.md`, and sweeps the whole tree for the old filename string and the `ADR OLD` phrase, rewriting both to the new number. It refuses to run against a dirty working tree (a failed run is then a plain `git checkout .`) and refuses a `NEW` that is already taken.

`OLD` names which file moves, and — this is the part that matters for the 0121/0125/0127/0128 shape specifically — it is not always a bare number. When two files claim the same `OLD`, a bare number cannot say which one the operator means, so the script refuses and lists both stems rather than silently picking one (`find | head -1`, the shape a first draft of this script shipped with and a fresh reviewer caught before merge). `OLD` also accepts the

filename's stem (`0128-a-settings-surface`) or the exact filename, either of which resolves to one specific file even when the number alone does not.

This does not prevent a collision — nothing about the numbering scheme changed. It changes the cost of the outcome the table above shows already happens regularly: today, resolving one costs a lane an hour of careful hand-editing across five kinds of file, which is exactly the setting the 0126 heading-drift bug entered from (the heading edit, one of five, silently didn't get staged). A single reviewable command with nothing left to forget removes the opportunity for that specific slip, whatever caused the original collision.

Tested two ways against a scratch clone of this repository's real corpus (never against this checkout — the script mutates files in place and a test run should never be the thing that renumbers a real ADR):

- **The unique-file case**, round-tripping a real ADR (`0001` → `9998` → `0001`): the forward run touched 20 files (the ADR itself, the README row, and 17 cross-references in other ADRs, specs, and Rust doc- comments — this ADR has no PDF-twin case since 0001 predates the tracked-PDF convention), `adr_index_matches_the_files` passed clean at the intermediate state, and the reverse run restored the tree to a byte-identical `git status --porcelain` (empty) against the starting commit.

- **The actual collision shape** — two files sharing one number, the case the round trip above does not exercise — built as a fixture (two real ADR bodies copied under `9997-test-collision-a.md` and `9997-test-collision-b.md`, both headed `# ADR 9997`). A bare `9997` was refused with both stems listed; `9997-test-collision-a` moved only that file; the sibling still legitimately claiming `9997` was left with its filename AND its own heading untouched. That last check is not incidental: an earlier version of the cross-reference sweep matched the literal phrase `ADR 9997` tree-wide with no exclusion, which is exactly the phrase both colliding files' own headings contain

— it correctly fixed dangling references elsewhere and incorrectly rewrote the sibling's own H1 to the new number while its filename stayed at the old one, replacing the resolved collision with a fresh instance of the 0126 bug this same script exists to make cheap to fix. The sweep now excludes any other `docs/adr/OLD-*.md` file from both patterns — a same-numbered sibling is a different document, not a reference to this one.

This is worth naming on its own terms. A collision-resolver bug whose failure mode is exactly the defect class this ADR's own new heading test exists to catch is not a coincidence worth glossing over: it is evidence about what kind of bug hides from reading and shows up under execution. Two independent readers (a cross-family review and this session's own re-check) read the sweep's exclusion logic before the fixture existed and neither found it — the code looked right, because the failure only appears when a specific *state* exists (two files sharing a number, both headed with the phrase the sweep searches for) that no amount of reading the sweep's logic in isolation reveals. Building the fixture and running the script against it is what found it. That is the argument for treating a fixture with a genuine two-file collision as the bar for proving this script works, not the unique-file round trip — a bar this ADR's own first draft fell short of before review.

Alternatives considered

Reserve a number on branch creation — a lane claims its number in a small, immediately-pushed commit to `main` (or a shared reservations file) before doing any of the actual ADR work. Rejected for this issue: it is the most complete fix, since it removes the window rather than narrowing what falls through it, but it requires every lane's workflow to change (an extra push-and-wait step before work can start) and a reservation-conflict story of its own (two lanes racing to reserve is the same race one

level up, just faster). Worth reconsidering if collisions keep recurring after this ADR's two mitigations ship — the table above would then be evidence that "make lateness cheap" was not enough, and that argument belongs in a future ADR that supersedes this one.

Drop sequential numbering for date-and-slug filenames (`2026-09-06-a-credential-exists-only-before-untrusted-checkout.md`, already `git-vista`'s convention for handoff files). Rejected: it has no contention by construction, which is the strongest guarantee on offer, but it is a bigger and much less reversible change than this issue warrants — it touches the index format, 129 existing files' cross- reference style (`[ADR 0126]`, `docs/adr/0126-...`, "ADR 0126" in prose across specs and Rust doc-comments), and the ordering the index currently gives for free from the filename alone. A repo-wide rename of this shape is itself exactly the kind of large mechanical change `scripts/adr-renumber.sh`'s cross-reference sweep was built to make safe — if this path is chosen later, that script is most of the tooling it would need.

Accept collisions as permanent and never fix the underlying process. Rejected: the table shows five in one day: at that rate, hand-resolving each at five-places-by-hand cost is the more expensive steady state, and it is exactly the cost decision 2 removes.

Consequences

- `adr_index_matches_the_files.rs` now has four tests instead of three; all four ran clean against the full 129-file corpus before this ADR was filed under number 130.
- `scripts/adr-renumber.sh` is now the documented path for resolving any future collision from the table's shape (0121/0125/0127/0128) — a same-number collision between two different files is still caught by the existing `files_by_number()` panic, and now has a one-command fix instead of a five-place hand-edit.

- Neither change requires any lane to alter its workflow before writing an ADR — the only step change is *after* a collision is found, not before one might happen. This is deliberate: it ships today rather than asking every concurrent lane to adopt a new pre-work ritual first.
- The reservation-on-branch-creation alternative remains open. If collisions recur at a similar rate after this lands, that recurrence is itself the evidence for revisiting it.

Mutation proof

`failure-atlas`'s `mutation_check` was unavailable for this issue — every call (against this repository and, checked independently, against another lane's unrelated worktree) returned `clone_failed: ContainmentError: temp base /tmp is inside the git work tree at /tmp`, a server-side regression unrelated to this change, reported upstream the same day. Per the standing exception for when the registered tool cannot run, both arms below were proved by hand: baseline green, mutation applied, test rerun and confirmed red, mutation reverted via a byte-for-byte backup, baseline green again — recorded here instead of a `mutation_check` id.

Both arms mutate `docs/adr/0001-repository-generation.md` and `docs/adr/0002-versioned-api-contract.md` (temporarily, reverted after each), targeting `every_adr_heading_names_its_own_filename_number`:

arm	mutation	mutated result
wrong but recognisable number	`# ADR 0001 —` → `# ADR 0002 —`	caught: fails on `filename says 0001, heading says 0002` — the "digits present, disagree" branch
no recognisable number at all	`# ADR 0002 —` → `#` (leading digits removed entirely)	caught, by a **different** assertion: fails on `heading has no recognisable number` — the "heading matches neither known style" branch, which the first arm never reaches

Both caught, neither survived; the two arms exercise disjoint branches of the same function (`Some(h) if h == filename_number` vs. `None`), so neither could have passed by accidentally tripping the other's assertion. Both files were restored from a pre-mutation copy and reconfirmed byte-identical (`git status --porcelain` empty) before this ADR was written.

`heading_numbers_by_file_number()`'s own map insert also gained the same duplicate-number panic `files_by_number()` already had, so an `--exact` run of only the heading test cannot silently drop one of two same-numbered files and report a false pass — a gap a fresh review of this PR found before merge, alongside the collision-fixture finding decision 2 records.